Summary of Topics Covered Exercises:15/10/2026.

Topic	Exercises Practicing It
Basic loops & control	Pattern printing exercises
Functions	Linked list search, reverse
Pointers & dynamic memory	All data structure exercises
Singly linked list	Search, delete, count, reverse, swap
Doubly linked list	Insert, delete, traverse, reverse
Circular linked list	Traverse, insert, delete, search
Stacks (LIFO)	Stack implementations
Queues (FIFO)	Queue implementations

PART 1:

• Write a C program to create a function to search an element in linked list. If element exists in the linked list then, it should return its index otherwise -1.

Answer:

Walk through the list from the head, counting the index. Return the index when **data** equals the key; if the end (NULL) is reached, return -1.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
```

```
}

int search(struct node *head, int key)
{
    int index = 0;
    while (head != NULL) {
        if (head->data == key)
            return index;           /* found: return its index */
        head = head->next;
        index++;
    }
    return -1;                      /* not found */
}

int main()
{
    int n, key;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    display(head);
    printf("Element to search: ");
    scanf("%d", &key);
    printf("Index = %d\n", search(head, key));
    return 0;
}
```

- Write a C program to print square/rectangle star(*) pattern.

Answer:

Use two nested loops: the outer loop prints the rows, the inner loop prints the stars in each row. A square is a rectangle where rows = columns. Both programs are given below.

– Print square star(*) pattern series of N rows.

Answer:

Outer loop runs N times, inner loop also runs N times.

```
#include <stdio.h>

int main()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {          /* rows */
        for (j = 1; j <= n; j++)        /* columns = rows */
            printf("* ");
        printf("\n");
    }
    return 0;
}
```

– Print rectangle star(*) pattern of N rows and M columns.

Answer:

Outer loop runs N times (rows), inner loop runs M times (columns).

```
#include <stdio.h>

int main()
{
    int n, m, i, j;
    printf("Enter rows (N) and columns (M): ");
    scanf("%d %d", &n, &m);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= m; j++)
            printf("* ");
        printf("\n");
```

```
    }
    return 0;
}
```

• Write a C program to print the Floyd's triangle number pattern using loop.

Answer:

Floyd's triangle has 1 number in row 1, 2 numbers in row 2, and so on. A counter **num** starts at 1 and increases after every print, so numbers continue across rows.

```
#include <stdio.h>

int main()
{
    int rows, i, j, num = 1;
    printf("Enter number of rows: ");
    scanf("%d", &rows);
    for (i = 1; i <= rows; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d ", num);
            num++;
        }
        printf("\n");
    }
    return 0;
}
```

• Write a C program to print hollow right triangle star pattern series using for loop.

Answer:

Print a star only on the first column (j == 1), on the diagonal (j == i) and on the last row (i == n). Everything else is a space.

```
#include <stdio.h>

int main()
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            /* print * on the first column, the diagonal and the last row */
            if (j == 1 || j == i || i == n)
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

• Write a C program to create a singly linked list of n nodes and delete the first node (beginning node).

Answer:

Save the head in **temp**, move head to the second node, then free temp.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
```

```
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *deleteFirst(struct node *head)
{
    struct node *temp;
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }
    temp = head;
    head = head->next;              /* second node becomes the head */
    free(temp);
    return head;
}

int main()
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Before: ");  display(head);
    head = deleteFirst(head);
    printf("After : ");  display(head);
    return 0;
}
```

- Write a C program to create a singly linked list of n nodes and delete the last node of the list.

Answer:

Travel to the last node while remembering the node before it (**prev**). Set prev->next = NULL and free the last node. Special case: a list with one node.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
```

```
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *deleteLast(struct node *head)
{
    struct node *cur, *prev = NULL;
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }
    if (head->next == NULL) {       /* only one node */
        free(head);
        return NULL;
    }
    cur = head;
    while (cur->next != NULL) {     /* go to the last node */
        prev = cur;
        cur = cur->next;
    }
    prev->next = NULL;              /* second last becomes the last */
    free(cur);
    return head;
}

int main()
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Before: ");  display(head);
    head = deleteLast(head);
    printf("After : ");  display(head);
    return 0;
}
```

- Write a C program to create a singly linked list of n nodes and delete a node from the middle of the linked list.

Answer:

Stop at the node *before* the position to delete, link it to the node after the deleted one, then free the deleted node.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *deleteMiddle(struct node *head, int pos)
{
    struct node *prev, *temp;
    int i;
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }
    if (pos == 1) {                 /* position 1 = first node */
        temp = head;
        head = head->next;
        free(temp);
        return head;
    }
    prev = head;
    for (i = 1; i < pos - 1 && prev != NULL; i++)
        prev = prev->next;          /* stop at the node BEFORE pos */
    if (prev == NULL || prev->next == NULL) {
        printf("Invalid position\n");
        return head;
    }
    temp = prev->next;              /* node to delete */
    prev->next = temp->next;        /* bypass it */
    free(temp);
    return head;
}

int main()
{
    int n, pos;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Before: ");  display(head);
```

```
    printf("Position to delete (middle): ");
    scanf("%d", &pos);
    head = deleteMiddle(head, pos);
    printf("After : ");  display(head);
    return 0;
}
```

PART 2.

- Write a C program to create a list of nodes and delete all nodes of the given linked list.

Answer:

Visit each node: keep it in **temp**, move head to the next node first, then free temp. Repeat until head is NULL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *deleteAll(struct node *head)
{
    struct node *temp;
    int count = 0;
    while (head != NULL) {
        temp = head;                /* remember the current node */
        head = head->next;          /* move on BEFORE freeing */
        free(temp);
        count++;
    }
    printf("%d nodes deleted\n", count);
    return NULL;                    /* list is now empty */
}

int main()
```

```
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    display(head);
    head = deleteAll(head);
    display(head);                  /* prints: NULL */
    return 0;
}
```

– How to delete entire linked list in C.

Answer:

1. Create a pointer **temp**.
2. While head != NULL: set temp = head.
3. Move head = head->next (do this BEFORE freeing, otherwise the link to the rest of the list is lost).
4. free(temp).
5. When the loop ends, head is NULL, which means the list is empty.

– How to clear all nodes of a linked list.

Answer:

Clearing a list means freeing every node and setting the head to NULL. Same logic as above:

```
while (head != NULL) {
    temp = head;
    head = head->next;
    free(temp);
}
/* head is now NULL: the list is empty */
```

- Write a C program to create a linked list and delete all nodes by key.

Answer:

First delete all matching nodes at the head (the head changes). Then scan the rest of the list: if the next node has the key, bypass it and free it; otherwise move forward.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
```

```
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *deleteByKey(struct node *head, int key)
{
    struct node *temp, *cur;
    while (head != NULL && head->data == key) {   /* key at the head */
        temp = head;
        head = head->next;
        free(temp);
    }
    cur = head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->next->data == key) {             /* key in next node */
            temp = cur->next;
            cur->next = temp->next;
            free(temp);
        } else
            cur = cur->next;
    }
    return head;
}

int main()
{
    int n, key;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    display(head);
    printf("Key to delete: ");
    scanf("%d", &key);
    head = deleteByKey(head, key);
    display(head);
    return 0;
}
```

– How to delete elements by key of a linked list in C program.

Answer:

1. While the head holds the key: move head forward and free the old head.
2. Set cur = head.
3. While cur and cur->next exist: if cur->next->data == key, set temp = cur->next, cur->next = temp->next, free(temp) (do not move cur).
4. Otherwise move cur = cur->next.
5. Return the (possibly new) head.

- Write a C program to create a singly linked list of nodes and count total number of nodes in given linked list.

Answer:

Traverse from head to NULL, adding 1 to a counter for every node.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int countNodes(struct node *head)
{
    int count = 0;
    while (head != NULL) {
        count++;
        head = head->next;
    }
    return count;
}

int main()
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    display(head);
    printf("Total number of nodes = %d\n", countNodes(head));
    return 0;
}
```

- Write a C program to create a singly linked list and reverse the order of nodes of the given linked list.

Answer:

Use three pointers (prev, cur, next) and turn every link to point backward. No data is moved.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
```

```
struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *reverse(struct node *head)
{
    struct node *prev = NULL, *cur = head, *next;
    while (cur != NULL) {
        next = cur->next;           /* 1. save the next node */
        cur->next = prev;           /* 2. reverse the link */
        prev = cur;                 /* 3. move prev forward */
        cur = next;                 /* 4. move cur forward */
    }
    return prev;                    /* prev is the new head */
}

int main()
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Original: ");  display(head);
    head = reverse(head);
    printf("Reversed: ");  display(head);
    return 0;
}
```

– How to reverse a singly linked list in C.

Answer:

Reverse the direction of every link, one node at a time. Start with prev = NULL and cur = head. In each step: save cur->next, make cur->next point to prev, move prev to cur, and move cur to the saved next node. When cur becomes NULL, prev is the new head. Time O(n), extra space O(1).

– Algorithm and steps to reverse a singly linked list.

Answer:

1. START. Set prev = NULL, cur = head.

2. If cur == NULL, the list is empty: stop.
3. Repeat while cur != NULL:
   a) next = cur->next b) cur->next = prev c) prev = cur d) cur = next
1. Set head = prev.
2. Display the list. END.

- Write a C program to create a linked list and accept two node positions from user. The program should swap two nodes in linked list without swapping their data.

Answer:

Find the two nodes and the nodes before them. Change the **next** pointers of the previous nodes, then swap the **next** pointers of the two nodes. The data inside the nodes is never touched. The head is updated if the first node is swapped.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createList(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    return head;
}

void display(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *swapNodes(struct node *head, int p1, int p2)
{
    struct node *prev1 = NULL, *x = head, *prev2 = NULL, *y = head, *t;
    int i;
    if (p1 == p2)
        return head;
    if (p1 > p2) {                  /* make sure p1 < p2 */
        i = p1; p1 = p2; p2 = i;
    }
    for (i = 1; x != NULL && i < p1; i++) {      /* find node at p1 */
        prev1 = x;
        x = x->next;
    }
    for (i = 1; y != NULL && i < p2; i++) {      /* find node at p2 */
```

```
        prev2 = y;
        y = y->next;
    }
    if (x == NULL || y == NULL || p1 < 1) {
        printf("Invalid position\n");
        return head;
    }
    if (prev1 != NULL)              /* relink the previous nodes */
        prev1->next = y;
    else
        head = y;
    prev2->next = x;
    t = x->next;                    /* swap the next pointers */
    x->next = y->next;
    y->next = t;
    return head;
}

int main()
{
    int n, p1, p2;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Before: ");  display(head);
    printf("Enter the two positions: ");
    scanf("%d %d", &p1, &p2);
    head = swapNodes(head, p1, p2);
    printf("After : ");  display(head);
    return 0;
}
```

- Write a C program to implement Doubly linked list data structure.

Answer:

A doubly linked list node has three fields: **prev**, **data**, **next**. This menu-driven program implements insertion at the end, forward display, backward display and count.

```
#include <stdio.h>
#include <stdlib.h>

struct dnode {
    int data;
    struct dnode *prev;
    struct dnode *next;
};

struct dnode *makeNode(int val)
{
    struct dnode *p = (struct dnode *)malloc(sizeof(struct dnode));
    if (p == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    p->data = val;
    p->prev = NULL;
    p->next = NULL;
    return p;
}

struct dnode *createList(int n)
{
    struct dnode *head = NULL, *tail = NULL, *newNode;
    int i, val;
    for (i = 1; i <= n; i++) {
        printf("Enter data of node %d: ", i);
        scanf("%d", &val);
        newNode = makeNode(val);
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
```

```
            newNode->prev = tail;
            tail = newNode;
        }
    }
    return head;
}

void displayForward(struct dnode *head)
{
    printf("NULL <-> ");
    while (head != NULL) {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

void displayBackward(struct dnode *head)
{
    struct dnode *cur = head;
    if (cur == NULL) {
        printf("NULL\n");
        return;
    }
    while (cur->next != NULL)       /* go to the last node */
        cur = cur->next;
    printf("NULL <-> ");
    while (cur != NULL) {           /* walk back using prev */
        printf("%d <-> ", cur->data);
        cur = cur->prev;
    }
    printf("NULL\n");
}

struct dnode *insertEnd(struct dnode *head, int val)
{
    struct dnode *newNode = makeNode(val), *cur = head;
    if (head == NULL)
        return newNode;
    while (cur->next != NULL)
        cur = cur->next;
    cur->next = newNode;
    newNode->prev = cur;
    return head;
}

int countNodes(struct dnode *head)
{
    int c = 0;
    while (head != NULL) {
        c++;
        head = head->next;
    }
    return c;
}

int main()
{
    struct dnode *head = NULL;
    int choice, val;
    do {
        printf("\n1.Insert at end  2.Display forward  3.Display backward");
        printf("  4.Count  0.Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1: printf("Value: ");
                scanf("%d", &val);
                head = insertEnd(head, val);
                break;
        case 2: displayForward(head);   break;
        case 3: displayBackward(head);  break;
        case 4: printf("Nodes = %d\n", countNodes(head)); break;
        case 0: break;
        default: printf("Invalid choice\n");
        }
```

```
    } while (choice != 0);
    return 0;
}
```

- Write a C program to create a doubly linked list and display all nodes of the created list.

Answer:

Create n nodes linking both directions (tail->next = new, new->prev = tail). Display forward from head; display backward by going to the last node and following **prev**.

```
#include <stdio.h>
#include <stdlib.h>

struct dnode {
    int data;
    struct dnode *prev;
    struct dnode *next;
};

struct dnode *makeNode(int val)
{
    struct dnode *p = (struct dnode *)malloc(sizeof(struct dnode));
    if (p == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    p->data = val;
    p->prev = NULL;
    p->next = NULL;
    return p;
}

struct dnode *createList(int n)
{
    struct dnode *head = NULL, *tail = NULL, *newNode;
    int i, val;
    for (i = 1; i <= n; i++) {
        printf("Enter data of node %d: ", i);
        scanf("%d", &val);
        newNode = makeNode(val);
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
    return head;
}

void displayForward(struct dnode *head)
{
    printf("NULL <-> ");
    while (head != NULL) {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

void displayBackward(struct dnode *head)
{
    struct dnode *cur = head;
    if (cur == NULL) {
        printf("NULL\n");
        return;
    }
    while (cur->next != NULL)
        cur = cur->next;
    printf("NULL <-> ");
```

```
    while (cur != NULL) {
        printf("%d <-> ", cur->data);
        cur = cur->prev;
    }
    printf("NULL\n");
}

int main()
{
    int n;
    struct dnode *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Forward : ");   displayForward(head);
    printf("Backward: ");   displayBackward(head);
    return 0;
}
```

– How to create and display a doubly linked list in C.

Answer:

Define the node with struct dnode { int data; struct dnode *prev, *next; }. For each new node (malloc) set prev = NULL, next = NULL and read data. If the list is empty, the node is head and tail. Otherwise tail->next = new, new->prev = tail, tail = new. To display, start at head and print while moving with **next** until NULL; for reverse display start at tail and move with **prev**.

– Algorithms to create and traverse doubly linked list.

Answer:

Create:

1. Read n. Set head = tail = NULL.
2. Repeat n times: allocate newNode, read data, newNode->prev = newNode->next = NULL.
3. If head == NULL: head = tail = newNode. Else: tail->next = newNode; newNode->prev = tail; tail = newNode.

Traverse:

1. If head == NULL: print “List is empty” and stop.
2. Set temp = head.
3. While temp != NULL: print temp->data; temp = temp->next.
4. (Backward: start at tail, while temp != NULL: print, temp = temp->prev.)

PART 4:

• Write a C program to create a doubly linked list and insert a new node in beginning, end or at any position in the list.

Answer:

Three functions: insertBeginning, insertEnd and insertAt. When inserting, always set both the **prev** and **next** links of the new node and fix the neighbours.

```
#include <stdio.h>
#include <stdlib.h>

struct dnode {
    int data;
    struct dnode *prev;
    struct dnode *next;
};
```

```
struct dnode *makeNode(int val)
{
    struct dnode *p = (struct dnode *)malloc(sizeof(struct dnode));
    if (p == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    p->data = val;
    p->prev = NULL;
    p->next = NULL;
    return p;
}

struct dnode *createList(int n)
{
    struct dnode *head = NULL, *tail = NULL, *newNode;
    int i, val;
    for (i = 1; i <= n; i++) {
        printf("Enter data of node %d: ", i);
        scanf("%d", &val);
        newNode = makeNode(val);
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
    return head;
}

void displayForward(struct dnode *head)
{
    printf("NULL <-> ");
    while (head != NULL) {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct dnode *insertBeginning(struct dnode *head, int val)
{
    struct dnode *newNode = makeNode(val);
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode;
    return newNode;                 /* new node is the new head */
}

struct dnode *insertEnd(struct dnode *head, int val)
{
    struct dnode *newNode = makeNode(val), *cur = head;
    if (head == NULL)
        return newNode;
    while (cur->next != NULL)
        cur = cur->next;
    cur->next = newNode;
    newNode->prev = cur;
    return head;
}

struct dnode *insertAt(struct dnode *head, int pos, int val)
{
    struct dnode *cur = head, *newNode;
    int i;
    if (pos == 1)
        return insertBeginning(head, val);
    for (i = 1; i < pos - 1 && cur != NULL; i++)
        cur = cur->next;            /* node before the position */
    if (cur == NULL) {
        printf("Invalid position\n");
        return head;
    }
```

```
    newNode = makeNode(val);
    newNode->next = cur->next;
    newNode->prev = cur;
    if (cur->next != NULL)
        cur->next->prev = newNode;
    cur->next = newNode;
    return head;
}

int main()
{
    int n, choice, val, pos;
    struct dnode *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    displayForward(head);
    do {
        printf("\n1.Insert beginning  2.Insert end  3.Insert at position  0.Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);
        if (choice >= 1 && choice <= 3) {
            printf("Value: ");
            scanf("%d", &val);
        }
        if (choice == 1)
            head = insertBeginning(head, val);
        else if (choice == 2)
            head = insertEnd(head, val);
        else if (choice == 3) {
            printf("Position: ");
            scanf("%d", &pos);
            head = insertAt(head, pos, val);
        }
        displayForward(head);
    } while (choice != 0);
    return 0;
}
```

– How to insert a new node at beginning of a Doubly linked list.

Answer:

newNode->next = head; if head exists, head->prev = newNode; head = newNode (newNode->prev stays NULL).

– How to insert a new node at the end of a doubly linked list.

Answer:

If the list is empty the new node is the head. Otherwise move to the last node (cur->next == NULL), then cur->next = newNode and newNode->prev = cur.

– How to insert a new node at any position of a doubly linked list in C.

Answer:

Move to the node before the position (cur). Then: newNode->next = cur->next; newNode->prev = cur; if cur->next exists, cur->next->prev = newNode; cur->next = newNode. Position 1 uses insert at beginning.

– Algorithm to insert node in a doubly linked list.

Answer:

1. Create newNode and store the data.
2. If inserting at the beginning: newNode->next = head; head->prev = newNode (if head exists); head = newNode.

3. If inserting at the end: traverse to the last node; last->next = newNode; newNode->prev = last.
4. If inserting at position p: traverse to node p-1 (cur); newNode->next = cur->next; newNode->prev = cur; cur->next->prev = newNode (if it exists); cur->next = newNode.
5. If the position is invalid (cur == NULL), print “Invalid position”.

- Write a C program to create a doubly linked list and delete a node from beginning, end or at any position of the linked list.

Answer:

Three functions: deleteBeginning, deleteEnd and deleteAt. Always connect the neighbours before freeing the node.

```
#include <stdio.h>
#include <stdlib.h>

struct dnode {
    int data;
    struct dnode *prev;
    struct dnode *next;
};

struct dnode *makeNode(int val)
{
    struct dnode *p = (struct dnode *)malloc(sizeof(struct dnode));
    if (p == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    p->data = val;
    p->prev = NULL;
    p->next = NULL;
    return p;
}

struct dnode *createList(int n)
{
    struct dnode *head = NULL, *tail = NULL, *newNode;
    int i, val;
    for (i = 1; i <= n; i++) {
        printf("Enter data of node %d: ", i);
        scanf("%d", &val);
        newNode = makeNode(val);
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
    return head;
}

void displayForward(struct dnode *head)
{
    printf("NULL <-> ");
    while (head != NULL) {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct dnode *deleteBeginning(struct dnode *head)
{
    struct dnode *temp;
    if (head == NULL) {
        printf("List is empty\n");
```

```
        return NULL;
    }
    temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
    return head;
}

struct dnode *deleteEnd(struct dnode *head)
{
    struct dnode *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }
    while (cur->next != NULL)
        cur = cur->next;            /* last node */
    if (cur->prev != NULL)
        cur->prev->next = NULL;
    else
        head = NULL;                /* it was the only node */
    free(cur);
    return head;
}

struct dnode *deleteAt(struct dnode *head, int pos)
{
    struct dnode *cur = head;
    int i;
    if (pos == 1)
        return deleteBeginning(head);
    for (i = 1; i < pos && cur != NULL; i++)
        cur = cur->next;            /* node to delete */
    if (cur == NULL) {
        printf("Invalid position\n");
        return head;
    }
    cur->prev->next = cur->next;
    if (cur->next != NULL)
        cur->next->prev = cur->prev;
    free(cur);
    return head;
}

int main()
{
    int n, choice, pos;
    struct dnode *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    displayForward(head);
    do {
        printf("\n1.Delete beginning  2.Delete end  3.Delete at position  0.Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);
        if (choice == 1)
            head = deleteBeginning(head);
        else if (choice == 2)
            head = deleteEnd(head);
        else if (choice == 3) {
            printf("Position: ");
            scanf("%d", &pos);
            head = deleteAt(head, pos);
        }
        displayForward(head);
    } while (choice != 0);
    return 0;
}
```

– How to delete a node from beginning of a doubly linked list.

Answer:

temp = head; head = head->next; if head exists, head->prev = NULL; free(temp).

– How to delete a node from end of a doubly linked list.

Answer:

Travel to the last node (cur->next == NULL). If cur->prev exists, set cur->prev->next = NULL; otherwise the list becomes empty (head = NULL). Then free(cur).

– How to delete a node from any position of a doubly linked list in C.

Answer:

Travel to the node at the position (cur). Set cur->prev->next = cur->next; if cur->next exists, set cur->next->prev = cur->prev; then free(cur). Position 1 uses delete from beginning.

– Algorithm to delete a node from doubly linked list.

Answer:

1. If head == NULL: print “List is empty” and stop.
2. Find the node to delete (cur): the head for the beginning, the last node for the end, or the node at the given position.
3. If cur is the head: head = cur->next; if head exists, head->prev = NULL.
4. Else: cur->prev->next = cur->next; if cur->next exists, cur->next->prev = cur->prev.
5. free(cur).

17. Write a C program to create a doubly linked list and reverse the linked list.

Answer:

Swap the **prev** and **next** pointers of every node. The last node becomes the new head.

```
#include <stdio.h>
#include <stdlib.h>

struct dnode {
    int data;
    struct dnode *prev;
    struct dnode *next;
};

struct dnode *makeNode(int val)
{
    struct dnode *p = (struct dnode *)malloc(sizeof(struct dnode));
    if (p == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    p->data = val;
    p->prev = NULL;
    p->next = NULL;
    return p;
}

struct dnode *createList(int n)
{
    struct dnode *head = NULL, *tail = NULL, *newNode;
    int i, val;
    for (i = 1; i <= n; i++) {
        printf("Enter data of node %d: ", i);
        scanf("%d", &val);
        newNode = makeNode(val);
        if (head == NULL)
            head = tail = newNode;
```

```
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
    return head;
}

void displayForward(struct dnode *head)
{
    printf("NULL <-> ");
    while (head != NULL) {
        printf("%d <-> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct dnode *reverse(struct dnode *head)
{
    struct dnode *cur = head, *temp = NULL;
    while (cur != NULL) {
        temp = cur->prev;           /* swap prev and next of this node */
        cur->prev = cur->next;
        cur->next = temp;
        cur = cur->prev;            /* old next node */
    }
    if (temp != NULL)
        head = temp->prev;          /* last processed node = new head */
    return head;
}

int main()
{
    int n;
    struct dnode *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createList(n);
    printf("Original: ");  displayForward(head);
    head = reverse(head);
    printf("Reversed: ");  displayForward(head);
    return 0;
}
```

– How to reverse the doubly linked list in C programming.

Answer:

Visit each node and swap its prev and next pointers (use a temporary pointer). Move to the next node using the old next, which is now stored in prev. When the loop ends, the new head is temp->prev, where temp is the last node processed. Time O(n), space O(1).

– Algorithm to reverse a doubly linked list.

Answer:

1. Set cur = head, temp = NULL.
2. While cur != NULL: temp = cur->prev; cur->prev = cur->next; cur->next = temp; cur = cur->prev.
3. If temp != NULL: head = temp->prev.
4. Display the list. END.

18.Write a C program to create a circular linked list of nodes and traverse the list.

Answer:

In a circular linked list the last node points back to the head, so there is no NULL. This program reads values until -1 is entered. To traverse, use a do-while loop and stop when we return to the head.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

/* reads values until the user types -1, then closes the circle */
struct node *createCircular()
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int val;
    printf("Enter data (-1 to stop): ");
    scanf("%d", &val);
    while (val != -1) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        newNode->data = val;
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
        printf("Enter data (-1 to stop): ");
        scanf("%d", &val);
    }
    if (tail != NULL)
        tail->next = head;          /* last node points to first */
    return head;
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);          /* stop when we are back at head */
    printf("(back to head)\n");
}

int main()
{
    struct node *head = createCircular();
    traverse(head);
    return 0;
}
```

– How to create a circular linked list of nodes and display all elements of the list in C.

Answer:

Create nodes one by one and link them like a singly linked list (keep head and tail). After the last node is added, set tail->next = head to close the circle. To display, start at head, print, move to next, and repeat until cur == head again (use do-while, because head itself must be printed first).

– Algorithm to create and traverse a Circular linked list.

Answer:

Create:

1. Set head = tail = NULL.
2. For each value: allocate newNode, set data. If head == NULL: head = tail = newNode. Else: tail->next = newNode; tail = newNode.
3. After the last node: tail->next = head.

Traverse:

1. If head == NULL: print “empty” and stop.
2. Set cur = head.
3. Do: print cur->data; cur = cur->next. While cur != head.

19. Write a C program to create a circular linked list of n nodes and traverse the list.

Answer:

Same idea, but the number of nodes n is entered first.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createCircular(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    if (tail != NULL)
        tail->next = head;          /* last node points back to first */
    return head;
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("(back to head)\n");
}

int main()
```

```
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createCircular(n);
    traverse(head);
    return 0;
}
```

– How to create a circular linked list of n nodes and display all elements of the list in C.

Answer:

Read n, then repeat the node creation n times (loop from 1 to n), adding each node at the tail. After the loop set tail->next = head. Display with a do-while loop that stops when cur == head.

– Algorithm to create and traverse a Circular linked list.

Answer:

1. Read n. Set head = tail = NULL.
2. Repeat i = 1 to n: create newNode, read data; if head == NULL: head = tail = newNode; else tail->next = newNode, tail = newNode.
3. tail->next = head.
4. Traverse: cur = head; do { print cur->data; cur = cur->next; } while (cur != head).

20. Write a program to create a circular linked list and insert a new node at the beginning or at any position in the given list.

Answer:

For insertion at the beginning, the last node must also point to the new head, so we first find the last node. For any position, we stop at the node before the position.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createCircular(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    if (tail != NULL)
        tail->next = head;          /* last node points back to first */
    return head;
```

```
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("(back to head)\n");
}

int countNodes(struct node *head)
{
    struct node *cur = head;
    int c = 0;
    if (head == NULL)
        return 0;
    do {
        c++;
        cur = cur->next;
    } while (cur != head);
    return c;
}

struct node *insertBeginning(struct node *head, int val)
{
    struct node *newNode = (struct node *)malloc(sizeof(struct node)), *last;
    if (newNode == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    newNode->data = val;
    if (head == NULL) {             /* first node points to itself */
        newNode->next = newNode;
        return newNode;
    }
    last = head;
    while (last->next != head)      /* find the last node */
        last = last->next;
    newNode->next = head;
    last->next = newNode;           /* keep the circle closed */
    return newNode;                 /* new head */
}

struct node *insertAtPosition(struct node *head, int pos, int val)
{
    struct node *cur = head, *newNode;
    int i, n = countNodes(head);
    if (pos < 1 || pos > n + 1) {
        printf("Invalid position\n");
        return head;
    }
    if (pos == 1)
        return insertBeginning(head, val);
    for (i = 1; i < pos - 1; i++)
        cur = cur->next;            /* node before the position */
    newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Memory not allocated\n");
        exit(1);
    }
    newNode->data = val;
    newNode->next = cur->next;
    cur->next = newNode;
    return head;
}

int main()
{
    int n, choice, val, pos;
```

```
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createCircular(n);
    traverse(head);
    do {
        printf("\n1.Insert at beginning  2.Insert at position  0.Exit\nChoice: ");
        scanf("%d", &choice);
        if (choice == 1) {
            printf("Value: ");
            scanf("%d", &val);
            head = insertBeginning(head, val);
        } else if (choice == 2) {
            printf("Value and position: ");
            scanf("%d %d", &val, &pos);
            head = insertAtPosition(head, pos, val);
        }
        traverse(head);
    } while (choice != 0);
    return 0;
}
```

– How to insert a new node at the beginning of a circular linked list in C.

Answer:

If the list is empty: newNode->next = newNode and it becomes the head. Otherwise find the last node (last->next == head), then newNode->next = head; last->next = newNode; head = newNode.

– How to insert a new node at any position in a circular linked list in C.

Answer:

Check that 1 ≤ position ≤ n+1. Position 1 is insertion at the beginning. Otherwise move to node position-1 (cur), then newNode->next = cur->next; cur->next = newNode. Because the list is circular, inserting after the last node still keeps the circle closed.

– Algorithm to insert new node in Circular linked list in C program.

Answer:

1. Create newNode and store the data.
2. If head == NULL: newNode->next = newNode; head = newNode; stop.
3. Beginning: find last (last->next == head); newNode->next = head; last->next = newNode; head = newNode.
4. Position p: move cur to node p-1; newNode->next = cur->next; cur->next = newNode.
5. Display the list.

21. Write a C program to delete element from circular linked list by key. In this article

Answer:

We search for the key starting at the head while remembering the previous node (it starts as the last node). When found, prev->next = cur->next, and if it was the head, the head moves forward. If it was the only node, the list becomes empty.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
```

```
};

struct node *createCircular(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    if (tail != NULL)
        tail->next = head;          /* last node points back to first */
    return head;
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("(back to head)\n");
}

struct node *deleteByKey(struct node *head, int key)
{
    struct node *cur, *prev;
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }
    prev = head;
    while (prev->next != head)      /* prev starts as the LAST node */
        prev = prev->next;
    cur = head;
    do {
        if (cur->data == key) {
            if (cur == head && cur->next == head) {   /* only node */
                free(cur);
                return NULL;
            }
            prev->next = cur->next;                   /* bypass cur */
            if (cur == head)
                head = cur->next;                     /* new head */
            free(cur);
            return head;
        }
        prev = cur;
        cur = cur->next;
    } while (cur != head);
    printf("Key %d not found\n", key);
    return head;
}

int main()
{
    int n, key;
    struct node *head;
```

```
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createCircular(n);
    traverse(head);
    printf("Key to delete: ");
    scanf("%d", &key);
    head = deleteByKey(head, key);
    traverse(head);
    return 0;
}
```

– You will explain how to delete by key element from circular linked list.

Answer:

1. If the list is empty, stop.
2. Find the last node and set prev = last, cur = head.
3. Do: if cur->data == key, go to step 4; else prev = cur, cur = cur->next. While cur != head.
4. If cur is the only node: free it and set head = NULL. Otherwise prev->next = cur->next; if cur == head, head = cur->next; free(cur).
5. If the loop ends without a match, print “Key not found”.

– You will explain logic and program to delete element from circular linked list by key.

Answer:

Logic: deleting means bypassing the node so that the previous node points to the next node, and then freeing it. The two special cases are (1) deleting the head, where the last node must also be re-linked to the new head, and (2) deleting the only node, where the list becomes empty. The full program is given in the answer above.

PART 5.

22. Write a C program to count nodes in Circular Linked List.

Answer:

Because there is no NULL, use a do-while loop that counts nodes until we return to the head.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createCircular(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
```

```
            tail = newNode;
        }
    }
    if (tail != NULL)
        tail->next = head;          /* last node points back to first */
    return head;
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("(back to head)\n");
}

int countNodes(struct node *head)
{
    struct node *cur = head;
    int count = 0;
    if (head == NULL)
        return 0;
    do {
        count++;
        cur = cur->next;
    } while (cur != head);
    return count;
}

int main()
{
    int n;
    struct node *head;
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createCircular(n);
    traverse(head);
    printf("Total elements = %d\n", countNodes(head));
    return 0;
}
```

– you will explain how to count nodes in Circular Linked List.

Answer:

Set count = 0 and cur = head (if head is NULL the answer is 0). Do: count++, cur = cur->next. While cur != head. The final value of count is the number of nodes. A normal while (cur != NULL) loop would never end, because a circular list has no NULL.

– You will use program C to create a Circular Linked List

Answer:

The program above uses the function createCircular(n): it creates n nodes with malloc, links them one after another and finally sets tail->next = head to close the circle.

– You'll count total elements in Circular Linked List.

Answer:

The function countNodes(head) in the program above returns the total number of elements (for 1, 2, 3 it prints: Total elements = 3).

23. Write a C program to create a Circular Linked List of n nodes and search for an element in it (search by key).

Answer:

Traverse with a do-while loop from the head, comparing each node with the key. Return the position (starting at 1) when found, or -1 when we come back to the head without finding it.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createCircular(int n)
{
    struct node *head = NULL, *tail = NULL, *newNode;
    int i;
    for (i = 1; i <= n; i++) {
        newNode = (struct node *)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory not allocated\n");
            exit(1);
        }
        printf("Enter data of node %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
            head = tail = newNode;
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }
    if (tail != NULL)
        tail->next = head;          /* last node points back to first */
    return head;
}

void traverse(struct node *head)
{
    struct node *cur = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("(back to head)\n");
}

int searchByKey(struct node *head, int key)
{
    struct node *cur = head;
    int pos = 1;
    if (head == NULL)
        return -1;
    do {
        if (cur->data == key)
            return pos;             /* position starting from 1 */
        cur = cur->next;
        pos++;
    } while (cur != head);
    return -1;                      /* back at head: not found */
}

int main()
{
    int n, key, pos;
    struct node *head;
```

```
    printf("How many nodes? ");
    scanf("%d", &n);
    head = createCircular(n);
    traverse(head);
    printf("Key to search: ");
    scanf("%d", &key);
    pos = searchByKey(head, key);
    if (pos == -1)
        printf("%d not found\n", key);
    else
        printf("%d found at position %d\n", key, pos);
    return 0;
}
```

24. Write a C program to implement the stack data structure using an array with push and pop operations.

Answer:

A stack is LIFO (Last In, First Out). **top** starts at -1. Push: check overflow (top == MAX-1), then top++ and store. Pop: check underflow (top == -1), then read stack[top] and top--.

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;                       /* -1 means the stack is empty */

void push(int value)
{
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = value;
    printf("%d pushed\n", value);
}

void pop()
{
    if (top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    printf("%d popped\n", stack[top]);
    top--;
}

void display()
{
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (top to bottom): ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main()
{
    int choice, value;
    do {
        printf("\n1.Push  2.Pop  3.Display  0.Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1: printf("Value: ");
                scanf("%d", &value);
                push(value);
                break;
```

```
        case 2: pop();      break;
        case 3: display();  break;
        case 0: break;
        default: printf("Invalid choice\n");
        }
    } while (choice != 0);
    return 0;
}
```

25. Write a C program to implement the stack data structure using a linked list with push and pop operations.

Answer:

The top of the stack is the head of the list. Push inserts a node at the beginning; pop removes the first node. There is no fixed size limit.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *top = NULL;            /* NULL means the stack is empty */

void push(int value)
{
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Stack Overflow (no memory)\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;            /* new node points to old top */
    top = newNode;                  /* new node becomes the top */
    printf("%d pushed\n", value);
}

void pop()
{
    struct node *temp;
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    temp = top;
    printf("%d popped\n", temp->data);
    top = top->next;                /* move top down */
    free(temp);
}

void display()
{
    struct node *cur = top;
    if (cur == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (top to bottom): ");
    while (cur != NULL) {
        printf("%d ", cur->data);
        cur = cur->next;
    }
    printf("\n");
}

int main()
{
    int choice, value;
    do {
        printf("\n1.Push  2.Pop  3.Display  0.Exit\nChoice: ");
```

```
        scanf("%d", &choice);
        switch (choice) {
        case 1: printf("Value: ");
                scanf("%d", &value);
                push(value);
                break;
        case 2: pop();      break;
        case 3: display();  break;
        case 0: break;
        default: printf("Invalid choice\n");
        }
    } while (choice != 0);
    return 0;
}
```

26. Write a C program to implement the queue data structure using an array, including enqueue and dequeue operations.

Answer:

A queue is FIFO (First In, First Out). Insert at the **rear** (enqueue) and remove from the **front** (dequeue). Both start at -1; when the last element is removed they are reset to -1.

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;          /* -1 means the queue is empty */

void enqueue(int value)
{
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;                  /* first element inserted */
    rear++;
    queue[rear] = value;
    printf("%d enqueued\n", value);
}

void dequeue()
{
    if (front == -1) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d dequeued\n", queue[front]);
    if (front == rear)
        front = rear = -1;          /* queue became empty: reset */
    else
        front++;
}

void display()
{
    int i;
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue (front to rear): ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main()
{
    int choice, value;
    do {
        printf("\n1.Enqueue  2.Dequeue  3.Display  0.Exit\nChoice: ");
```

```
        scanf("%d", &choice);
        switch (choice) {
        case 1: printf("Value: ");
                scanf("%d", &value);
                enqueue(value);
                break;
        case 2: dequeue();  break;
        case 3: display();  break;
        case 0: break;
        default: printf("Invalid choice\n");
        }
    } while (choice != 0);
    return 0;
}
```